Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Системы обработки информации и управления» (ИУ5)

ОТЧЕТ

по лабораторной работе № 2

«Объектно-ориентированные возможности языка Python»

по дисциплине
«ПиКЯП»

Выполнил:
студент группы ИУ5-33Б _____ Тютюнник Н.В.
(подпись)
«___» _____ 2026 г.

Проверил:
преподаватель кафедры ИУ5 _____ Гапанюк Ю.Е.
(подпись)
«___» _____ 2026 г.

2026

1 Цель работы

Изучение объектно-ориентированных возможностей языка Python: абстрактных классов, наследования, композиции, свойств и методов класса. Разработка консольной программы для вычисления площадей геометрических фигур и проверка её работы модульными тестами.

2 Задание

Разработать консольное приложение на Python 3 по заданию лабораторной работы № 2. Номер варианта по списку группы N = 23. Требуется:

1. Создать виртуальное окружение, установить через pip внешний пакет и вызвать одну из его функций в программе.

2. Разместить исходные файлы, кроме main.py, в пакете lab_python_oop; каждый класс описать в отдельном модуле.

3. Описать абстрактную геометрическую фигуру с абстрактным методом площади и отдельный класс цвета со свойством.

4. Реализовать прямоугольник и круг как наследников фигуры. Хранить цвет через объект класса цвета; площадь круга вычислять с math.pi.

5. Реализовать квадрат как наследника прямоугольника. Для трёх фигур задать название полем класса, получать его методом класса и определить __repr__ с использованием format.

6. В main.py создать синий прямоугольник 23 × 23, зеленый круг радиуса 23 и красный квадрат со стороной 23, вывести параметры и площади. Запуск оформить через проверку __name__.

7. Выполнить дополнительное задание: проверить программу модульными тестами.

Источник задания: github.com/ugapanyuk/BKIT_2022/wiki/lab_python_oop.

3 Описание решения

3.1 Структура классов

Shape наследуется от ABC. Метод area помечен @abstractmethod, поэтому создать экземпляр Shape нельзя. Метод класса get_name возвращает поле NAME соответствующего наследника.

Класс Color хранит цвет в поле _value. Свойство value с декораторами @property и @value.setter позволяет читать и изменять цвет, отклоняя пустую строку и значения других типов. Создание отдельного объекта Color в конструкторе фигуры реализует композицию.

Rectangle и Circle наследуют Shape и переопределяют area. Square наследует Rectangle, передаёт одинаковую сторону в super().__init__ и использует унаследованный метод площади. Каждый класс находится в отдельном модуле пакета lab_python_oop.

3.2 Вычисление площади и вывод

Формулы площади: $S = width \times height$ для прямоугольника, $S = \pi \times radius^2$ для круга, $S = side^2$ для квадрата. При N = 23 получаем 529, $529\pi \approx 1661{,}902513749$ и 529. Метод __repr__ возвращает параметры, название, цвет и площадь через format; {:.2f} округляет только вывод.

При создании фигуры статический метод _positive отклоняет нулевые и отрицательные размеры, бесконечность и NaN с исключением ValueError.

3.3 Виртуальное окружение и тестирование

Создано окружение .venv, через pip установлен colorama 0.4.6. В main вызывается init(autoreset=True), константы Fore задают цвет строк. При перенаправлении вывода ANSI-коды удаляются. Версия зависимости закреплена в requirements.txt.

Модуль test_shapes.py содержит 10 тестов unittest для проверки классов фигур, ошибочных значений и вывода для варианта 23.

4 Текст программ

Листинг 1 – Описание пакета (файл __init__.py)

```
"""Классы геометрических фигур для лабораторной работы № 2."""
```

Листинг 2 – Абстрактная фигура (файл shape.py)

```
from abc import ABC, abstractmethod
from math import isfinite



class Shape(ABC):
    NAME = "Геометрическая фигура"

    @classmethod
    def get_name(cls):
        return cls.NAME

    @staticmethod
    def _positive(value):
        if not isfinite(value) or value <= 0:
            raise ValueError("Размер должен быть положительным и конечным")
        return value

    @abstractmethod
    def area(self):
        """Вычислить площадь фигуры."""
        raise NotImplementedError
```

Листинг 3 – Цвет фигуры (файл color.py)

```
class Color:
    def __init__(self, value):
        self.value = value

    @property
    def value(self):
        return self._value

    @value.setter
    def value(self, value):
        if not isinstance(value, str) or not value.strip():
            raise ValueError("Цвет должен быть непустой строкой")
        self._value = value.strip()
```

Листинг 4 – Прямоугольник (файл rectangle.py)

```
from .color import Color
from .shape import Shape



class Rectangle(Shape):
    NAME = "Прямоугольник"
```

```
    def __init__(self, width, height, color):
        self.width = self._positive(width)
        self.height = self._positive(height)
        self.color = Color(color)

    def area(self):
        return self.width * self.height

    def __repr__(self):
        return (
            "{}: ширина={:g}, высота={:g}, "
            "цвет={}, площадь={:.2f}"
        ).format(self.get_name(), self.width, self.height,
                 self.color.value, self.area())
```

Листинг 5 – Круг (файл circle.py)

```
import math

from .color import Color
from .shape import Shape



class Circle(Shape):
    NAME = "Круг"

    def __init__(self, radius, color):
        self.radius = self._positive(radius)
        self.color = Color(color)

    def area(self):
        return math.pi * self.radius ** 2

    def __repr__(self):
        return "{}: радиус={:g}, цвет={}, площадь={:.2f}".format(
            self.get_name(), self.radius, self.color.value,
            self.area())
```

Листинг 6 – Квадрат (файл square.py)

```
from .rectangle import Rectangle



class Square(Rectangle):
    NAME = "Квадрат"

    def __init__(self, side, color):
        super().__init__(side, side, color)

    def __repr__(self):
        return "{}: сторона={:g}, цвет={}, площадь={:.2f}".format(
            self.get_name(), self.width, self.color.value,
            self.area())
```

Листинг 7 – Основная программа (файл main.py)

```
from colorama import Fore, init

from lab_python_oop.circle import Circle
from lab_python_oop.rectangle import Rectangle
from lab_python_oop.square import Square



N = 23



def main():
    init(autoreset=True)
    print("Вариант: {}".format(N))
    figures = (
        (Rectangle(N, N, "синий"), Fore.BLUE),
        (Circle(N, "зеленый"), Fore.GREEN),
        (Square(N, "красный"), Fore.RED),
    )
    for figure, console_color in figures:
        print(console_color + repr(figure))
    print("Внешний пакет: colorama; вызван init(autoreset=True).")



if __name__ == "__main__":
    main()
```

Листинг 8 – Модульные тесты (файл test_shapes.py)

```
import math
import unittest

from .circle import Circle
from .color import Color
from .rectangle import Rectangle
from .shape import Shape
from .square import Square



class ShapeTests(unittest.TestCase):
    def test_abstract_class(self):
        with self.assertRaises(TypeError):
            Shape()


    def test_rectangle_area(self):
        self.assertEqual(Rectangle(3, 7, "синий").area(), 21)


    def test_circle_area(self):
        self.assertAlmostEqual(Circle(2, "зеленый").area(),
                               4 * math.pi)


    def test_square_inheritance(self):
        square = Square(5, "красный")
        self.assertIsInstance(square, Rectangle)
        self.assertEqual((square.width, square.height), (5, 5))
        self.assertEqual(square.area(), 25)
```

```
    def test_class_names(self):
        for cls, name in ((Rectangle, "Прямоугольник"),
                          (Circle, "Круг"), (Square, "Квадрат")):
            with self.subTest(cls=cls):
                self.assertEqual(cls.get_name(), name)

    def test_color_property(self):
        color = Color("синий")
        color.value = " зеленый "
        self.assertEqual(color.value, "зеленый")

    def test_color_composition(self):
        first = Rectangle(2, 3, "синий")
        second = Rectangle(2, 3, "синий")
        self.assertIsInstance(first.color, Color)
        first.color.value = "красный"
        self.assertEqual(second.color.value, "синий")

    def test_invalid_color(self):
        for value in ("", "  ", None, 23):
            with self.subTest(value=value):
                with self.assertRaises(ValueError):
                    Color(value)

    def test_invalid_dimensions(self):
        for value in (0, -1, math.inf, math.nan):
            for build in (lambda: Rectangle(value, 1, "синий"),
                          lambda: Rectangle(1, value, "синий"),
                          lambda: Circle(value, "зеленый"),
                          lambda: Square(value, "красный")):
                with self.subTest(value=value, build=build):
                    with self.assertRaises(ValueError):
                        build()

    def test_variant_23_and_repr(self):
        cases = (
            (Rectangle(23, 23, "синий"), 529,
             "Прямоугольник: ширина=23, высота=23, "
             "цвет=синий, площадь=529.00"),
            (Circle(23, "зеленый"), 1661.9025137490005,
             "Круг: радиус=23, цвет=зеленый, площадь=1661.90"),
            (Square(23, "красный"), 529,
             "Квадрат: сторона=23, цвет=красный, площадь=529.00"),
        )
        for figure, expected_area, expected_repr in cases:
            with self.subTest(figure=figure):
                self.assertAlmostEqual(figure.area(), expected_area)
                self.assertEqual(repr(figure), expected_repr)


if __name__ == "__main__":
    unittest.main()
```

Листинг 9 – Внешняя зависимость (файл requirements.txt)

```
colorama==0.4.6
```

5 Результаты выполнения программы

На рисунке 1 показаны версия Python и установленный пакет в окружении .venv. Установка выполнена через pip с файлом lab_python_oop/requirements.txt.

```
$ .venv/bin/python --version
Python 3.12.14

$ .venv/bin/python -m pip list --no-cache-dir --disable-pip-version-check
Package  Version
-------- -------
colorama 0.4.6
pip      25.0.1
```

Рисунок 1 – Версия Python и внешний пакет в окружении .venv

При запуске main.py для варианта 23 получены площади 529, 1661,90 и 529. Последняя строка сообщает о вызове init из colorama (рисунок 2).

```
$ .venv/bin/python main.py
Вариант: 23
Прямоугольник: ширина=23, высота=23, цвет=синий, площадь=529.00
Круг: радиус=23, цвет=зеленый, площадь=1661.90
Квадрат: сторона=23, цвет=красный, площадь=529.00
Внешний пакет: colorama; вызван init(autoreset=True).
```

Рисунок 2 – Вывод программы для варианта 23

Запуск тестов завершился сообщением Ran 10 tests и OK (рисунок 3). Все проверки пройдены. На рисунках приведён фактический текст вывода команд.

```
$ .venv/bin/python -m unittest lab_python_oop.test_shapes
..........
----------------------------------------------------------------------
Ran 10 tests in 0.000s

OK
```

Рисунок 3 – Успешное выполнение 10 модульных тестов

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы разработано консольное приложение на Python 3 для работы с геометрическими фигурами. Реализованы абстрактный класс, наследование прямоугольника и круга, наследование квадрата от прямоугольника, композиция с объектом цвета, свойство для доступа к цвету и метод класса для получения названия фигуры.

Для варианта 23 получены площади 529 для прямоугольника и квадрата и 529π для круга. Методы __repr__ формируют строки с параметрами, цветом и площадью через format. В виртуальное окружение установлен colorama и выполнен вызов его функции init.

Дополнительное задание выполнено: разработаны и успешно пройдены 10 модульных тестов. Тестирование подтвердило правильность вычислений, строковых представлений, наследования, работы свойства цвета и обработки проверяемых некорректных данных.